

Next：以直接担保实现连续 UTXO 支付

Lu Hengrun

摘要—持续支付要求收款方在先前交易仍待处理时复用已收到的价值。Next 通过注册担保组织，在未花费交易输出 (UTXO) 账本中支持这一操作。收款方在验证并记录某个输出及其法定人数签名证书 (TXCer) 后，即可请求后继付款，而公共执行异步进行。若已认证的来源缺失，委员会原子地执行该后继付款，并记录一项由来源发行者备付金支持的义务；该义务在来源执行时关闭，或在到期后通过备付金扣款关闭，赔付之后到达的来源向赔付方发行者回款。在固定成员与拜占庭故障下，我们证明了消费唯一性、包含隐藏证书在内的负债覆盖，以及延迟中性：对于来源全部执行的有限无冲突付款集合，每种成功的来源到达与赔付顺序最终得到的 CAL 最终持有人和备付金余额，都与无延迟执行相同。受限的资金来源修订保持完整区块身份。在单台 M4 Max 主机上（九个服务、内存状态、NoSync 钱包），100 跳链到达最后一个已认证收款的中位时间为 158.846 ms，而每跳之间等待公共确认时为 65.204 s；三次各 30,000 笔的运行含排空完成 1,750–1,887 笔/秒。九次回款运行（磁盘委员会）关闭了 21,600 笔付款，并回收全部 216 笔赔付。

关键词—UTXO 支付，支付担保，拜占庭容错，授权修复。

I. 引言

持续支付要求收到的价值能在一连串转账中保持可用。在 UTXO 账本中，若让每个后继付款都等待其来源交易公共执行，就会在本已就绪的付款之间插入确认延迟。设 Alice 付款给 Bob，Bob 用收到的输出再付款给 Carol。Bob 需要在 Alice 的交易执行之前，就取得可支持新付款的证据。难点不仅是速度：一旦 Bob 的付款先执行，就必须有人为 Alice 的交易尚未交付的价值负责；而且无论哪笔交易先到达账本，乃至 Alice 的交易是否会到达，账务都必须保持正确。

基于证书的支付系统表明，收款方侧的可用性可以与部分公共处理分离。FastPay 与 Zef 支持低延迟的已认证支付，Sui Lutriss 则针对不同形式的状态访问结合了广播与共识路径 [1]–[3]。支付通道协议通过通道关系及其流动性提供快速转账 [4], [5]。我们针对原生账本中的一项具体执行需求：来源尚未执行的已认证输出应能支持一笔公共执行的后继付款，而缺失来源的责任

应落在一个已确定、且为此预留了资金的主体身上。在 Next 中，用户花费自己被路由的输出，注册组织为未决来源提供备付，交易双方之间无需预先注资的通道路径。

出现三个相互耦合的问题。第一，证书可能在公共登记之前就已流通，因此公共缺口记账会漏掉部分已有承诺；签名容量必须覆盖每一笔已认证的付款，包括隐藏的付款。第二，来源可能在委员会已为其缺失价值赔付之后才到达。迟到的来源不得对同一价值支付两次——既付给垫付它的担保方，又付给已经花掉它的收款方；否则延迟本身就会转移资金。第三，解决某个来源可能涉及一笔其输出已被消费的付款，因此对其所记录资金来源的任何更改，都必须保持其后继付款所引用的所有者授权、输出标识符和承诺。仅凭稳定的交易哈希，既不能规定也不能授权这样的更改。

Next 固定付款权利，同时把未决来源分配给其直接发行者。输出证书 (TXCer) 把付款的输出绑定到一个明确的发行组织。收款方在验证并原子地记录输出及其 TXCer 之后，可以通过该输出指定的消费组织请求后继付款，并携带直接输入证书作为祖先证据；所有者授权、花费状态、费用与资源限制则单独检查。

若某个已认证输入在公共创建状态中仍不存在，委员会原子地消费它，针对发行者记录一项直接义务，并创建子输出；即使义务尚未关闭，这些子输出也是最终的。来源成功执行即履行该义务。截止时间过后，公共赔付通过一次实际的备付金扣款关闭该义务，该扣款补足缺失的支撑，而不是再次向子收款方入账。若来源之后到达，它在同一转移中向垫付该价值的发行者回款，并且不为收款方创建第二个输出。当所有来源最终都执行时，延迟改变的是临时的资本敞口，而最终的本金分配保持不变。

经济责任关闭可以采用追加式记录。Next 在同一公共转移中联合提交赔付与有效的资金来源修订，因此赔付进展还要求取得三份适配贡献。记账不变量的强制检查独立于适配器给出的结果。修复接口在既有承诺下

安装受限的资金来源替换，保留实际备付扣款的历史表示。标准钱包、同步与重放读取原始命令；修订后的正文由修复构造、安装和本地检查读取。我们把设计组织为三项职责各异的贡献。

C1: 面向一轮续花的分层架构。我们把成员认证、收款方交付、证据复制和公共执行分开。成员在签名前原子地锁定输入并预留容量，一轮并行认证产生 TXCer，安装与提交在后台进行。注册组织在组织内与跨组织使用同一证据接口，付款本金与担保方备付金相互分离。

C2: 直接责任与有资金支持、延迟中性的账务。我们定义原子的缺失输入转移及其正常、赔付和已回收三条关闭路径；对于固定的输入与证书形态，每个请求的祖先证据不随转账深度增长。在固定成员的拜占庭模型下，我们证明唯一消费、签名容量为每一张曾签发的证书（包括隐藏证书）的负债设定上界，以及资产守恒。我们还证明：对于来源全部执行的有限无冲突付款集合，任意的到达与赔付交错最终得到的 CAL 持有量和备付金余额，都与无延迟执行相同。

C3: 保持身份的授权表示更新。我们把资金来源的更改限制在经公共授权的位置，同时保持所有者授权、输出引用以及包括分片集合头在内的完整区块身份。同块批处理在按序应用经济步骤的同时共享最终分片适配，安装可直接推进到已提交的修订，而无需重复经济效果。该机制为修复构造者和安装者提供与承诺相容的历史表示，其适配工作量有界且可共享。

在单台 M4 Max 主机上，使用九个服务、内存服务状态和 NoSync 钱包，100 跳链到达最后一个已认证收款的三次运行中位时间为 158.846 ms，而每跳之间等待公共确认时为 65.204 s。三次各 30,000 笔的运行含后台排空完成 1,750–1,887 笔/秒。另有九次运行的回款矩阵关闭了 21,600 笔付款，并回收全部 216 笔赔付；成对执行保持了 CAL 的最终分布。第 III 节和第 IV 节定义设计；第 V 节和第 VI 节展开其安全性论证与评估。

II. 背景与相关工作

Next 连接基于证书的连续续花、备付支撑的公共执行及授权历史修复。最接近的工作已经在认证、抵押和受控编辑方面建立先例。我们依据缺失资金来源的责任归属，以及责任落实过程中保持的付款与区块身份，界定 Next 的研究定位。

A. 基于证书的支持与执行

FastPay 通过拜占庭一致广播实现低延迟支付，并支持由预充值资金支撑的主账本部署 [1]。其可兑付性论证允许在相关入账转账尚未全部在本地确认时确认有效证书，前提是补齐发送方先前转出付款的缺失证书。兑付提供了一条基于证书从主账本资金池获取资金的路径。Zef 将可弃置账户与具有隐私性的可转让币相结合。正常消费使用当前币证书及消费证据，而非完整的祖先交易历史 [2]。

Sui Lutris 对归属特定所有者的对象采用广播执行，对共享对象的交互采用共识 [3]。Stingray 通过拜占庭有界计数器和 FastUnlock 支持并发交易 [6]；其重叠额度与回收机制是与 Next 资源核算密切相关的先例。其可交换事务工作负载所处理的争用问题，与每笔交易消费前序输出的支付链有所不同。Next 关注在资金来源交易执行之前，公共执行对经认证 UTXO 的消费，同时登记由直接发行组织承担、具有资金保障的直接义务。全部输出金额的预留与累计损失的保留，既计入已经公共登记的责任，也计入隐藏证书所代表的承诺。

B. 通道与抵押担保

Blitz 使用 pay-or-revoke 替代通道支付中的两阶段提交，Donner 则构建应对 Domino 攻击的虚拟通道 [4], [5]。CryptoMaze 提供部分付款不可链接的原子多路径通道支付，同时避免在共享边上重复建立链下合约 [7]。这些机制基于通道安排运行。Next 使用具有已注册消费路由和组织备付资金的原生账本，因此交易双方既无需双边通道，也无需在彼此之间预先建立注入流动性的通道路径。

Snappy 通过抵押支持的批准机制保护接受快速链上付款的商户。可复用支付担保将隐私保护与可在规定时间窗口内重复使用的客户抵押相结合 [8], [9]。Next 则将用户付款本金与组织提供的额外担保资金分离。其赔付关闭已执行子付款的缺失资金来源，而不再次向该子付款的收款人增加本金。

HyperService 通过公共 Network Status Blockchain 与 Insurance Smart Contract，对跨链执行违约进行仲裁，并在资金层面撤销已提交交易的效果 [10]。Next 处理的是单个原生 UTXO 账本内不同的责任边界：赔付结清发行组织的直接义务，同时保持已执行后继付款的本金效果。

C. 受控编辑与历史一致性

变色龙哈希允许在同一摘要下存在经授权的替代消息，并为可编辑区块链设计提供基础 [11]。SDR 研究去中心化的基于属性的编辑权限与动态委托；Escaping 将委员会选择和投票移至共识层之外，以减少修订等待 [12], [13]。Reconstructing Chameleon Hash 提出更强的不可区分性与抗碰撞性定义 [14]。Next 采用已有适配原语，由公共支付状态检查提供实施兼容替换的授权。

Reparo 提供公开可验证的修复层，并可在修复状态时重新执行受影响的交易 [15]。Next 固定付款授权和输出：公共修复命令对赔付进行核算，历史物化则替换获准修改的资金来源表示，而不重新执行后继付款。原始命令和后续修复仍构成经济重放历史。

RedactChain 协调同块并发编辑冲突与预处理 [16]。CDEdit 使用可复用权限令牌，减少多次编辑中重复的令牌和密钥生成及验证工作 [17]。Next 的批处理在共享最终区块表示的同时，保留每项直接义务的资格、备付扣款和终态检查。其完整区块身份由稳定区块头哈希与原始分片集头共同构成。经公共提交的逻辑修订授权确切的替换正文；物理安装可直接推进到后续已授权修订。这一设计将仅生效一次的经济赔付与表示更新的物理安装分离，同时保持后继引用，并支持重放同一公共命令历史。

III. 系统模型与概述

Next 使收款方能够在创建某输出的交易尚未公共执行之前，花费这个新认证的输出。该设计把已认证的付款权利、其公共消费以及其资金来源的解决三者分开。表 I 汇总了符号。

表 I
主要符号。

符号	含义
G, C	担保组织与公共委员会
T, c_T	付款及其已认证的摘要事实
(x, i)	输出标识符与花费实例
g, B_g	授权额度与累计授权
a_T, p_T, Rec_T	CAL 输出总额、总赔付（毛额）以及已回收的赔付
$\mathcal{O}_x$	针对缺失输出的直接义务
(H_b, P_b)	稳定区块哈希与原始分片集合头
ρ_ℓ, ρ_p	已提交的逻辑修订与已安装的物理修订

A. 参与者、资产与信任

Next 运行一个原生 UTXO 账本。CAL 承载支付价值，FUEL 用于支付处理费用。用户花费自己的输出；组织在公共备付账户中提供额外的 CAL。使用快速路径需要有一个带受支持组织路由的输出，而不是把付款本金存入担保方的备付金，或开设双边通道。默认的费用来源是用户的最终 FUEL 输出；同时也支持经明确授权的组织代付模式。

每个组织有四名固定成员和一个可替换的收集器。成员独立验证请求并授权付款。收集器分发请求、汇集证书并协调传播，自身不具备独立的花费权限。另有独立的委员会 C ，由四名等权重的验证者组成，三票即可提交。它对公共命令排序，并维护账户、输出、义务和修复授权。

攻击者在整个执行期间至多控制每个组织中的一名成员和一名委员会验证者。腐化是静态的；用户和收集器可以任意行事。诚实参与者共享可信的网络配置、创世、规则和组织注册表。每条组织路由对应唯一的活动配置和一个消费锁定域。所有者与成员签名不可伪造；规范摘要与原始区块承诺满足所需的绑定性。修复使用门限适配，而应用某次修复的许可来自公共验证与提交。门限材料由可信的离线 dealer 生成并分发。

诚实的签名状态是原子的，既不被遗忘也不被回滚。公共执行是确定性的并原子地提交；跟随者认证连续的公共前缀。就安全性而言，消息投递不必及时。进展需要足够多响应的参与者、可得的证据、费用与资源，以及公平的投递与调度。实验用的内存配置提供运行期状态原子性，每次运行都从全新创世开始。

B. 输出、路由与证书

输出指明其资产、金额、所有者和消费路由。其标识符 x 由网络、创建它的交易及输出位置导出。花费以 (x, i) 索引，普通输出使用实例 $i = 0$ 。快速付款的每个输入都分配给同一个消费组织，并由其所有者授权新交易。新创建的输出可以选择另一个已注册的组织。因此，跨组织续花改变的是新输出的路由，而不转移先前发行者的责任。

TXCer 由一份摘要和一份法定人数证书 (QC) 组成：即三名不同成员对同一事实 c_T 的签名。摘要绑定网络、交易身份、发行者、配置、epoch、规则、输出承诺与金额，以及准入授权。输出正文可以单独交付；收款方在保存输出之前检查相应的承诺、位置和所有权。

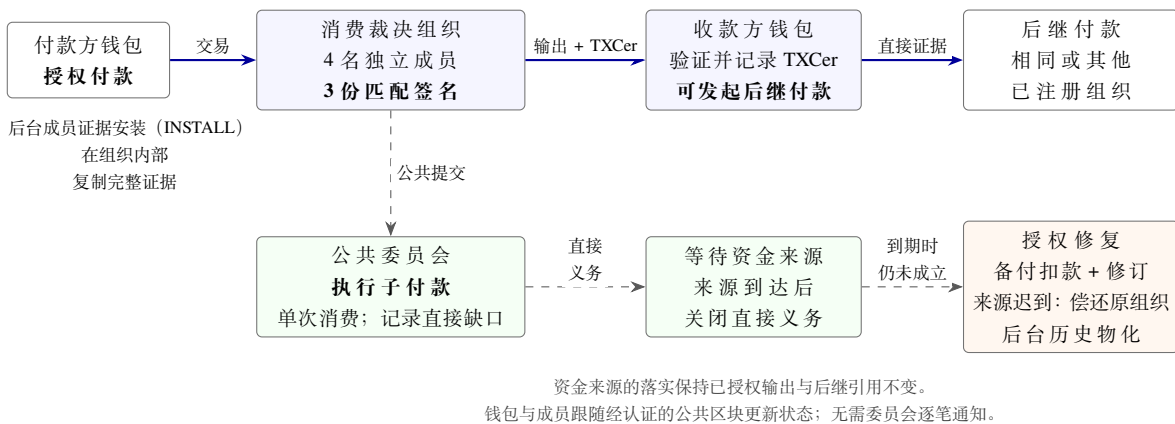


图 1. Next 中的连续支付与资金来源的公共落实。实线箭头表示收款人的续花路径，虚线箭头表示后台工作。每笔后继付款均需独立授权；尚未落实的资金来源由其直接发行组织承担责任，而不是作为等待祖先交易的要求向后传播。历史修复由后续公共命令授权；赔付后来源成功执行时偿还原赔付组织，不再向收款方重复入账。

该证书认证的是发行者的担保；它既不是额外的资产，也不是已经登记的公共义务。

我们区分已认证收款（钱包验证并在本地记录一个输出）、公共执行（一条已提交的命令消费输入并创建输出）和负债关闭（缺失的来源被履行或被赔付）。收款为后续请求提供证据；该请求要经过各自的授权与资源检查。实现中的 **Settled** 标志表示公共执行成功，并可与非零的 **Pending** 共存。下文中的来源到达指来源交易的公共执行成功，而不是收到其字节。

C. 三层设计

认证与续花。成员在签名前原子地预留输入实例及适用的资源。CAL 授权额度 g 授权 B_g ；每名成员获得 $\lfloor 2B_g/3 \rfloor$ 的签名容量，并在其本地执行 Worker 之间排他地划分。一笔付款预留 a_T ，即其所有 CAL 输出之和，含找零。这些相互重叠的签名容量约束的是承诺，而不是创造额外的备付金。收集器在一轮并行请求-响应中收到三份匹配的批准后返回 TXCer。收款方验证并原子地本地记录即完成快速路径；保存完整的中继材料、成员安装（INSTALL）和公共提交在后台进行。输入锁独立于预算释放而持续存在。

与来源解耦的公共执行。公共命令携带交易、准入向量、组织授权和直接输入证书。委员会重建已认证的摘要并验证授权，无需单独的新输出证书封装。它还检查当前的消费状态。当某个已认证输入尚未被公开创建时，委员会可以原子地记录其消费、针对发行者登记整张证书的覆盖、创建 O_x 及相应的缺口，并创建子输出。登记对证书的所有输出只做一次；只有实际被消费的缺失输

出才产生直接义务。执行使用直接证据，而不是递归地收集祖先交易正文。

钱包、成员和收集器各自独立地跟随已认证的区块与结果。成员依据自己记录的借记恢复容量。在观察到自己的付款成功执行后，成员恢复其 CAL 借记，仅保留其为该付款的输出所记录的净赔付，并只应用相对于先前恢复量的增量。由于来源的执行会在同一转移中为该付款所有已赔付的输出回款，此时这一净额为零。因此备付金损失只在其来源尚未执行期间存在。创世时按每个备付账户汇总并检查授权额度。增额从受保护备付集合之外的账户扣款，并原子地增加目标余额与授权额度；执行期间授权永不减少。

赔付、回款与历史修复。缺失的义务通过来源到达关闭，或在由公共区块时间确定的截止时间之后，由其发行者的备付金赔付关闭。赔付之后到达的来源在自己的执行中向赔付方发行者回款。修复把经济变更与对指定 **Funding/opening** 的经过验证的替换及其修订任务一起提交。受限的变色龙哈希适配保持固定的授权、输出引用以及完整区块身份 (H_b, P_b) 。RepairBatch 把指向同一历史区块的义务分组，并在一个原子事务内顺序应用它们的经济效果。

原始命令历史、已提交的逻辑修订 ρ_ℓ 和已安装的物理修订 ρ_p 相互独立。安装可以直接推进到经授权的较新版本，而无需重复赔付。经济重放使用原始命令及其后的修复；既有的后继付款既不需要新的签名，也不需要重新执行。

D. 续花示例与设计目标

Alice 经由 G_A 向 Bob 支付 100 CAL, 创建了路由到 G_B 的输出 x 。Bob 验证其 TXCer, 并获得一张针对 Carol 付款的证书。若 Bob 的付款先执行, 委员会消费 $(x, 0)$, 针对 G_A 记录一项 100 CAL 的义务, 并创建 Bob 付款的输出。Carol 可以由这笔付款继续, 无需获得 Alice 的交易正文。

若 Alice 的付款在赔付之前成功执行, 它关闭该义务, 而不会让已消费的实例 0 复活。一旦公共时间到达截止时间, 仍为 Open 的义务可以改由一次经授权的修复关闭: 该修复从 G_A 扣款, 并替换 Bob 付款中的资金引用; Carol 不会得到第二次本金入账。若 Alice 的付款在赔付之后执行, 它按相同规则消费自己的输入, 并向 G_A 偿还其垫付的 100 CAL。不会为 Bob 创建第二个输出, 因为他的付款已经获得资金, 且实例 $(x, 0)$ 保持已消费。若 Alice 的付款始终不执行, 则 G_A 承担该损失。

该设计面向在公共执行之前的续花、有资金支持的直接义务, 以及来源解决过程中对既有付款的保持。它把收款方的已认证证据与有界的公共责任及可审计的修复路径联系起来, 同时使历史安装不进入常规付款的等待条件。

IV. 协议设计

Next 认证固定的付款授权和输出, 同时允许其来源保持未决。收款方在授权后继付款时把 TXCer 用作证据。公共执行原子地消费缺失的已认证输入, 并记录其直接发行者义务。经授权的修复把义务的经济关闭与一个精确的替换资金表示一起提交。历史安装在该提交之后进行, 且从不重新执行该付款。

A. 身份、认证与交付

令 K_T 表示编码后的付款正文与输入声明, C_T 表示其输入承诺, Σ_T 表示其输出摘要:

$$\begin{aligned} K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\ \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\ C_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)). \end{aligned} \quad (1)$$

所有者签名认证 $\text{id}(T)$, 但不进入其计算。每个输出标识符绑定该身份、网络及其输出索引。资金引用与 opening 单独编码, 因而允许受限替换, 而不改变已授权的声明或输出。初始提交要求每个被声明的输入都有引用它的

OriginalFunding 以及有效的 opening。其后的修复保持正文、声明、承诺和所有者签名不变。因此静态验证缓存以完整命令字节为键, 而不把相同的交易身份视为可互换的验证结果。

对于一次新的批准, 成员检查所有权、路由、直接输入证书、价值守恒和准入向量。在一个本地事务中, 它重新检查当前输入锁、最终费用输入和指定 Worker 的资源切片, 然后记录批准及精确的借记。CAL 预留等于 a_T (含找零), 即使所有本金输入都已是最终的: 因为每个新认证的输出都可能在该付款执行之前流通。成员在事务提交之后签名。重试复用该批准; 释放资源容量绝不会清除其消费锁。

收集器把一个规范请求发送给四名成员, 并在验证了对同一 c_T 的三份签名后返回。不同的有效签名子集保持相同的事实身份。收款方对照输出正文、索引、所有者和可信配置检查 TXCer, 然后原子地记录该币。它可以立即构造后继请求。每一跳提供自己的直接输入证书, 而不是嵌入祖先交易链。

完整的中继材料在保存的同时, 有界的早发任务允许 INSTALL 与公共提交独立进行。安装在本地记录已认证的消费, 而不创建另一次批准或预算借记。成员相对于其首次安装安排备用提交; 重复安装不能推迟该期限。HTTP 接受使发件箱保持待决。经认证的公共完成会将其移除, 并优先于迟到的保存。该传播路径依赖完整的提交材料对持续发送者保持可得。

公共的 DirectSubmission 包含交易、准入向量、组织 SpendQC 以及所需的直接输入证书。委员会重建 Σ_T 并验证同一授权。省略单独的新输出 TXCer 封装改变的是信封, 而不是授权要求。

B. 含直接义务的原子执行

公共执行把整张证书的覆盖与实际的缺失输入义务分开。首次出现对某父事实 c 的缺失使用时, register 预留其全部已认证的 CAL 金额, 并为每个输出创建一个 Promise。覆盖记录维持

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

其中 Paid_c 是累计的毛额计数器。另一个计数器 $\text{Rec}_c \leq \text{Paid}_c$ 记录由来源执行回款的部分。授权额度的使用须满足 $\text{Reserved}_g + \text{Spent}_g \leq B_g$, 其中 Spent_g 是 g 之下各证书 $\text{Paid}_c - \text{Rec}_c$ 之和。登记会增加公共授权使用量, 但不从备付账户扣款。一次有效的赔付从账户扣

Require: 已验证的提交 T 、当前状态 S 、公共时间

- 1: 若 T 已经 settled, 则返回无新效果
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: 检查意图、授权额度与工作量; 在 O 中为费用托管注资
- 4: **for** 每个 CAL 输入 (x, i) **do**
- 5: 要求 (x, i) 未被消费
- 6: **if** 存在 (x, i) 的最终创建记录 **then**
- 7: 对照声明检查其正文与证据
- 8: **else**
- 9: 要求 $i = 0$ 且有匹配的直接 TXCer c
- 10: 若 c 的覆盖尚未登记, 则登记其全部覆盖
- 11: 创建唯一的 O_x ; 增加缺口与 Pending
- 12: **end if**
- 13: 在 O 中记录 (x, i) 的消费
- 14: **end for**
- 15: 要求 CAL 输入与输出总值相等
- 16: 记录 T 为 settled; 推进其费用阶段
- 17: 解决每个输出: 履行 Open 义务; 对已赔付的输出向发行者回款并标记为 Recovered; 否则履行任何已登记的 Promise。除非已回收, 否则创建最终实例 0
- 18: 把全部变更与结果作为一个转移返回

图 2. 公共付款执行。任何检查失败都会拒绝全部临时变更。输出的解决遵循第 IV-C 节所述分支。

款并增加 Spent_g ; 回款向同一账户入账, 并把 Spent_g 减少相同金额。两者都不改变 B_g 。只有实际被消费的缺失输出才产生直接义务并增加公共缺口。因此, 消费一张 40/60 证书中缺失的 40-CAL 输出, 会预留 100 CAL, 但只产生 40-CAL 的缺口。未使用的 Promise 仍被覆盖。执行不会仅因为自己的新输出已被认证, 就为它们登记覆盖。

图 2 给出了公共转移。其前状态包括同一区块中先前成功的命令。静态授权可以缓存; 消费、余额、授权额度和义务则对照当前状态评估。所有检查与变更都在私有 Overlay 中进行。失败会丢弃整个转移, 包括临时费用与覆盖。

每项义务绑定被消费的输出、父证书事实、发行者、金额、消费交易及历史输入位置。其截止时间以公共区块时间为锚。消费该输入的付款记录其 Pending 输入义务和费用进度。用户付费的费用消费经认证的最终 FUEL, 把多余的输入价值作为找零返还, 并把已签署的最大值托管。该最大值涵盖登记、结算、关闭, 以及每个已认证输入一次修复费用。阶段与义务标识符防止重复收费; 未用的托管在其输入义务关闭时退还给原付款方。

该转移使子输出成为最终输出, 即使某些输入仍是义务。后续交易可以按常规的最终输入检查消费这些输出。未决的责任留在直接认证的来源处, 而不是作为递归的执行前提向上传播。

C. 来源关闭、回款与累计对账

来源交易通过正常的付款执行到达, 包括验证并消费它自己的输入。随后, 它的每个输出都根据其义务的状态, 在来源自己的转移内得到解决。若义务为 Open, 到达会履行其 Promise、解除相关覆盖, 并减少消费者的 Pending 计数。若输出从未被消费, 已登记的任何 Promise 会被履行, 该输出成为最终实例 0。若该义务已被赔付, 则消费者当初是由备付金扣款提供资金的, 因此到达时偿还该扣款, 而不是创建第二个输出: 同样的金额被记入支付它的发行者账户, 义务变为 Recovered, 回款计数器与授权额度的 Spent 同时更新。在每个分支中, 先前已消费的实例 0 都保持已消费。已回收的输出没有最终创建记录, 不能再次被主张。回款不增加任何授权, 因为它归还的是赔付所耗用的容量。来源自己的输入可能引入新的直接义务; 它们针对各自的直接发行者准入, 并在同一个原子付款转移内检查。不追踪祖先, 回款也不能被撤销。

成员根据已认证的连续区块与结果重建进度。只有已有的本地批准才提供可恢复的借记。对其 CAL 借记, 成员 m 计算

$$t_{m,T} = \begin{cases} a_T - (\hat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{已结算,} \\ 0, & \text{其他情形,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

其中“已结算”表示成员已观察到自己付款成功执行; $\hat{p}_{m,T}$ 是为该批准的输出在本地记录的赔付, $\widehat{\text{Rec}}_{m,T}$ 是在本地记录的回款。在观察到自己付款的执行时, 成员检查经认证结果中报告的回款覆盖 $\hat{p}_{m,T}$, 并令 $\widehat{\text{Rec}}_{m,T} = \hat{p}_{m,T}$: 该次执行偿还 T 的每一个已赔付输出, 因此成员记录的损失被全额回款。只在跟随赔付之后才批准 T 的成员没有记录任何赔付 ($\hat{p}_{m,T} = 0$), 除自己的借记外不恢复任何东西。在检查边界与单调性之后, 成员把 $\delta_{m,T}$ 在原 Worker 中从 Reserved 转到 Available, 并更新 Applied。该付款自己的 Pending 输入不推迟 CAL 释放: 这些负债由其发行者承担。工作资源要等待费用关闭; 实际花掉的 FUEL 仍保持被扣除。重复的区块和迟到的材料不能重复入账。尚未执行的来

源不会仅因其部分输出已被赔付而自动获得释放。钱包把已回收的输出记录为终态观察，而不是币：它没有证书、没有最终创建记录、没有余额，旧 TXCer 的后续副本也不会覆盖它。

一次经授权的备付增额从外部的、不受保护的 CAL 账户扣款，向备付金入账，并原子地增加其授权额度。每名成员加上新旧 $[2B_g/3]$ 份额之差，而不重置 Reserved 或批准。汇总的备付检查覆盖共享同一账户的所有授权额度。

D. 构造并执行修复批

RepairBatch 针对一个历史区块，包含 1–32 个按规范排序的条目。每个条目给出一个输出标识符、交易索引、输入索引和新的输入 opening。义务与位置必须唯一。批固定基础修订，以及修订前后规范正文的摘要；成功使逻辑修订从 Base 推进到 Base + 1。金额、发行者和替换引用均由可信的义务导出，而不是作为可编辑的付款数据提供。

构造者读取最新已提交的逻辑表示，与物理安装无关。在请求适配份额之前，它对符合条件的义务排序，并在临时 Overlay 中依次模拟现有的赔付规则。成功应用的条目把其变更贡献到该模拟中。资格判定读取原始的已提交视图，而不是模拟的关闭记录。最终选定的集合再作为固定批检查一次。这些检查避免为已知会超出其账户或费用资源的批请求证明；它们不预留资金。

对每个条目，构造者由网络和被消费的输出导出一个备付金扣款标识，然后把其 OriginalFunding 位置替换为相应的备付引用和 opening。同一交易中的多个位置被打补丁到同一份副本中。序列化长度和所有非目标字节保持不变，包括先前经授权的替换。输入适配逐位置进行。最终组合后的正文决定受影响的区块分片，每个分片只适配一次；未变化的分片保留其承诺与证明。

份额端点绑定到固定的委员会身份。参与者独立地重建并从经济上检查所请求的固定批。收集器以实际的密码学组合来测试有界的三成员候选，只有当每个受影响分片都验证通过时才接受一个候选。格式错误或无效的响应排除的是候选，而不是来自其他成员的三份有效贡献。成功组合后取消尚未完成的请求。opening 确立承诺相容性；公共执行提供修复授权。

公共执行对照其当前状态重复所有检查（图 3）。在同一个 Overlay 内顺序评估处理共享账户和费用托管：

Require: 批 B 、状态 S 、执行高度与时间

- 1: $b \leftarrow \text{BatchID}(B)$
- 2: 若成功的 Task b 已存在，则返回无新效果
- 3: 加载 Canonical；检查高度较旧、base、previous 摘要
- 4: 检查条目顺序，以及义务与位置的唯一性
- 5: 要求每项义务为 Open、已到期且位置匹配
- 6: 重建所允许的组资金替换
- 7: 要求规范候选正文摘要等于 $B.\text{Next}$
- 8: 验证非目标字节、长度和完整区块身份
- 9: $O \leftarrow \text{Overlay}(S)$ ；初始化结果
- 10: **for** 按规范顺序的每个条目 **do**
- 11: 对照 O 评估赔付；要求 Applied
- 12: 把变更合并入 O ；追加效果与费用输出
- 13: **end for**
- 14: 记录共享的逐义务完成标识
- 15: 向 O 添加一个逻辑修订、Task 和队列项
- 16: 返回转移及供提交的执行结果

图 3. 原子的修复批执行。失败会拒绝整个批；只有成功的转移才被提交。批身份涵盖完整的规范命令，包括其证明数据。

最后一个被关闭的输入可以完成其消费者的费用并产生退款。基础修订变化、条目已关闭或资源不足会拒绝整个批。精确的成功重放在基础修订验证之前即被识别；不同的批与兼容的单项修复入口共享义务终态，从而防止通过重新打包再次扣款。在固定的前状态、时间和条目顺序下，批处理把相同的逐义务经济步骤组合成一个转移；命令与修订标识则仍特定于所选的打包方式。

经认证的结果包含逐条目效果（标明父事实与消费者事实、金额和输入位置）以及顶层的费用输出。成员原子地应用相关的逐条目效果及其游标，并定位原始批准以更新 Paid 与 Pending。钱包只应用每个费用输出一一次。赔付绝不会向已执行的后继付款再增加本金入账。回款不是批操作：当被批处理义务的来源到达时，每笔回款都在该来源自己的执行结果中报告，批身份不起作用。

E. 保持身份的安装与重放

输入承诺保持交易身份；分片承诺保持完整区块身份 (H_b, P_b) ，其中 P_b 包含分片数量与 Merkle 根。变化的分片获得指向其原始承诺的有效 opening。原始共识分片使用 opening 1 与修订 0。候选、锁和提交检查比较的是完整区块身份，而不只是稳定的头部哈希。

公共修订与 Task 授权一个精确的新正文及其分片证据。安装器在获取物理修订锁之前，从一个已提交的

读视图中复制该材料。对目标修订 r ：较低的物理版本经验证后安装；相等的版本要求材料完全相同；较高的版本涵盖旧任务，不回滚。原始区块被保留，分片与本地版本元数据原子写入。安装可以跳过中间的物理版本，同时保留每一条经济命令。

队列进度保持全局且连续。对于条目 A_1, B_1, A_2 ，在处理 A_1 时安装 A_2 并不完成 B_1 ；游标只在已处理的条目之间推进。逻辑执行读取已提交的 Canonical 修订，因此不同的安装调度留下相同的经济历史。重放使用原始区块及其后的修复命令，为同一公共命令序列重建应用哈希。既有的后继付款保留其授权、输出引用和区块身份。

追加式方案可以在不修订历史的情况下关闭经济责任。当前实现把赔付与有效修订一起提交，因此进展要求适配贡献可得；记账不变量不依赖适配器提出的结果。历史接口还提供另一项能力：修复之后，存储的区块携带一个资金表示，它指明为消费者垫付的备付金扣款，并仍能对照原始承诺验证。标准钱包、区块同步和重放读取原始命令；已提交的修订由修复构造、安装和本地检查读取。修订后的正文记录的是赔付时由哪个备付金为该输入提供资金，而不是现在由谁承担损失：之后的回款是来源结果中报告的另一项经济事实。批处理在区块内共享表示工作，直接安装合并历史写入；原始命令与逻辑修订仍被保留。这两个步骤都不会给常规认证增加一轮，也不要求收款方等待历史安装。

补充方法从能力与依赖条件两方面对比这一表示与追加式赔付方案。

V. 安全性分析

我们分析第 IV 节协议的有限执行。每个固定的四成员组织以及独立的、等权重的四验证者委员会，至多各有一个静态拜占庭参与者；证书需要三个不同的签名。诚实参与者共享合法的创世和一个注册表，该注册表为每条路由指定唯一的配置和锁定域。签名状态是原子的，且从不被遗忘或回滚；应用转移是确定性且原子的，跟随者应用经认证的连续区块，状态与游标原子地更新。所有论断都针对这些提交边界处的状态。我们以签名不可伪造、规范摘要与原始分片承诺具有绑定性为条件。适配输出被视为不可信候选：一次修订只通过已提交的修复生效，其守卫导出允许变动的位置、金额与发行者，并验证全部非目标字节和完整区块身份；安装只接受与已提交 Task 正文逐字节相同的内容。门

限适配提供与承诺相容的候选，其可得性决定修复进展。延迟是任意的。我们先建立原始执行接口，再连接认证、备付覆盖与修复。共识论证把 Tendermint 的锁定接口 [18] 特化到完整区块身份。

A. 原始执行与已认证消费

polka 是同一轮中针对某个完整区块身份或 nil 的 prevote 法定人数。

引理 1 (原始执行一致性): 在共识接口的锁定与有效性规则下，每个已提交高度只接纳一个原始区块正文，包括其应用上下文。

证明: 在原始 opening 1 下，分片叶子哈希其与上下文绑定的消息代表元。绑定性与 Merkle 证明在原始分片集合头之下固定其字节。候选正文由其分片重建，或在本地与分片配对；Locked 与 Valid 的转移保持这一配对。Proposal、锁证明 (POL)、锁和提交的比较都使用 (H_b, P_b) ，更换下载目标会清除旧正文。匹配的当前轮 *polka* 可以在 Proposal 到达之前触发 precommit；已有的轮次与步骤守卫仍然阻止第二次 precommit。

同轮冲突提交需要一名诚实验证者 precommit 两次。对于第 r 轮中值 v 的一次提交，取其两名诚实签署者，并假设之后某轮存在冲突的 *polka*。选取最小的这样的轮次 $s > r$ 及其最早的证书形成。其法定人数与这两名签署者相交。该签署者的投票需要一个更早的解锁理由：更低轮次的冲突与最小性矛盾，而更早的同轮 *polka* 与所选的形成事件矛盾。这里包括授权解锁的 nil *polka*。该论证使用签名事件，而不是对已完成提交证书的观察。诚实的轮次单调推进，因此提交签署者在更高轮次的投票，跟随其自身更早的锁，即使提交证书是之后才汇集的。对锁定值所投之票的有效性可以追溯到更早的锁轮次，并最终追溯到实际的提案验证。Finalize 跟随与候选的完整区块身份相匹配的提交。原始同步与重放保持正文、高度和时间，因此重试提供相同的执行材料。 ■

引理 2 (经认证的唯一消费): TXCer 固定其输出与发行者。没有两张有效证书能授权对同一 (x, i) 的冲突消费，且公共执行至多消费该实例一次。

证明: 绑定性固定已认证的摘要和经所有者授权的交易。固定的路由把相互竞争的请求送到同一个组织。两个三成员法定人数至少相交于两名成员，其中包括一名诚实成员，其原子输入锁先于签名。该成员不能批准两个冲突。预算释放保持该锁。最终输入与证书输

入的执行查询同一个花费键；事实重放既不产生另一次消费，也不产生另一次费用收取。已回收的输出没有最终创建记录，其实例 0 保持已消费，因此之后任何对 (x, i) 的主张都不能成功。 ■

B. 覆盖公共承诺与隐藏承诺

对一个完整的 CAL 授权额度 g ，累计授权 B_g 给予每名成员 $s(B_g) = \lfloor 2B_g/3 \rfloor$ 的容量。令 C_g 包含截至当前前缀、其三份签名曾经存在的每一个不同事实。它包括未汇集的与隐藏的证书，且从不删除已结算的事实。唯一的 CAL 分配把每个事实绑定到一个配置、ResourceKey 和授权额度 ID。事实 c 有输出上限 a_c ，即其 CAL 输出之和。对 $c \in C_g$ ，令 p_c 为为这些输出累计已付的赔付， Rec_c 为来源执行回款的部分， z_c 表示 c 的付款已成功执行。净赔付为 $\bar{p}_c = p_c - \text{Rec}_c$ 。定义

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

引理 3 (来源回款): 已赔付的输出只会在其来源的成功执行内、在使 $z_c = 1$ 的同一转移中离开 Repaired 状态，且回款记入被扣款的账户。因此 $0 \leq \text{Rec}_c \leq p_c \leq a_c$ ，当 $z_c = 0$ 时 $\text{Rec}_c = 0$ ，而一旦 $z_c = 1$ 则 $\text{Rec}_c = p_c$ 。特别地，来源待执行期间 $\bar{p}_c = p_c$ ，之后 $\bar{p}_c = 0$ ，故来源执行之后不留下任何净赔付。

证明: 每个已登记的 promise 至多离开 Open 一次，且赔付只适用于有资金支持的 Open 义务，故 $p_c \leq a_c$ 。离开 Repaired 的唯一转移，是事实等于该义务的证书事实、且其发行者等于该义务的发行者的某个付款的输出解决步骤；它与把该付款记录为 settled 在同一个 overlay 中执行，因此 z_c 与回款一起变化。该步骤解决来源中每个有义务的输出：Open 变为 Fulfilled，Repaired 变为 Recovered。没有义务的输出不产生赔付。其后 c 的每个 promise 都已关闭，故不再会产生新的赔付，重复提交也是无操作。赔付借记与回款入账都使用义务中保存的发行者账户，它就是 c 的 CAL 分配所对应的账户。 ■

对任一本地批准，定义 $r_{m,c} = a_c - \text{Applied}_{m,c}$ ，其中 Applied 是其累计的 CAL 恢复量。

引理 4 (残余容量): 对每个事实 $c \in C_g$ 和批准了 c 的每名诚实成员 m ，在每个前缀上， $r_{m,c} \geq w_c$ 。

证明: 在 m 观察到 c 的付款成功执行之前，它为 c 恢复的 CAL 为零，故无论赔付状态如何， $r_{m,c} =$

$a_c \geq w_c$ 。在 m 观察到该执行之后， m 检查所报告的回款覆盖其记录的赔付，并令其记录的回款等于该赔付。恢复目标是 a_c 减去为零的记录净赔付，故 $r_{m,c} = 0$ ，而由引理 3， $w_c = \bar{p}_c = 0$ 。尚未观察到其付款公共执行的成员保持 $r_{m,c} = a_c \geq 0 = w_c$ 。该陈述涵盖在跟随某次赔付之后才批准的成员：它没有记录任何赔付，只恢复自己的借记。 ■

定理 1 (潜在负债有资金支持): 对每个 CAL 授权额度， $Q_g \leq B_g$ 。对每个备付账户 x ，其未关闭缺口总额 D_x 不超过其余额 A_x 。

证明: 令 $\mathcal{A}_{m,g}$ 包含 g 之下所有成功的批准，即使没有 QC，并令 $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$ ，跨 Worker 求和。批准与恢复在 Available 与 Reserved 之间转移容量。增额加上成员份额之差，故对本地观察到的授权 $B_{m,g}$ ，有 $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$ ；授权额度只增不减。选取该组织的任意固定的三名诚实成员 H_g 。一张 QC 有三个不同的签署者，且至多有一名成员在 H_g 之外，因此 H_g 中签署了 c 的成员集合 Sig_c 至少有两个元素。诚实成员只在批准之后签名，故由引理 4，对 $m \in \text{Sig}_c$ 有 $r_{m,c} \geq w_c$ ，并且

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3 \lfloor 2B_g/3 \rfloor \leq 2B_g. \end{aligned} \quad (5)$$

令 R_c 与 d_c 为已登记的剩余覆盖与已解除覆盖，登记之前 $p_c = d_c = R_c = 0$ 。已登记的记录满足 $a_c = p_c + d_c + R_c$ ，且 $z_c = 1$ 蕴含 $R_c = 0$ 。因此 $R_c \leq w_c - \bar{p}_c$ ：当 $z_c = 0$ 时 $\text{Rec}_c = 0$ ，故 $R_c \leq a_c - p_c = w_c - \bar{p}_c$ ；而一旦 $z_c = 1$ ，两边均为零。定义 $S_g = \sum \bar{p}_c$ 、 $V_g = \sum R_c$ 和 $E_g = Q_g - S_g$ ，求和范围为 C_g 。登记、赔付、回款与解除维持公共的 Usage.Spent = S_g 与 Usage.Reserved = V_g 。

令 $\mathcal{G}_x$ 包含所有由 x 支持的授权额度。它们所认证的 CAL 账户等于被扣款的发行者账户。创世建立 $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$ 。外部的、不受保护的增额使两边等量增加。赔付使 S_g 增加、 A_x 减少相同金额；由引理 3，回款使 S_g 减少、 A_x 增加相同金额。没有其他协议操作会提取 CAL 备付金。每项 Open 义务对应一个不同的剩余 promise，于是

$$\begin{aligned} D_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

毛额 Paid 及其回款各只计一次； E_g 是风险上界，而不是可用的成员容量。 ■

CAL 准入推论。在执行前状态下（包括同一区块中先前成功的命令），令 N_g 为 g 之下不同的、有效的、尚未登记的父事实，它们为匹配的、未被消费的缺失输入所需要。它们有 $w_c = a_c$ 且 $p_c = R_c = 0$ 。每个已登记的事实满足 $R_c + \bar{p}_c \leq w_c$ ，且各事实互不相交，故

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

对同一证书的重复使用只贡献一项，而不是每个输入一项。顺序登记会看到先前的预留，且每个部分和都在该界之内。因此首次登记不会仅因 CAL 授权容量而失败，即使它在同一转移中先于回款，因为回款只会降低 S_g 。同样，一笔 Open 金额 $a \leq D_x \leq A_x$ 有足够的备付金本金。其他资源与费用检查仍然单独进行。

C. 资产守恒与延迟中性

定理 2 (CAL 守恒): 令 U 为最终的、未花费的 CAL 实例总值， A 为 CAL 账户余额总和， D 为 Open 缺口总值。每个执行前缀都满足

$$U + A - D = C_0, \quad (8)$$

其中 C_0 包含创世输出和所有账户，包括增额的来源账户。

证明: 一笔付款消费最终价值 F 与缺失价值 M ，产生 $F + M$ 。令 J 为其到达的、义务为 Open 的输出， R 为其义务已被赔付的输出。 J 中的输出已被消费，故它们关闭缺口而不增加未花费价值； R 中的输出没有创建记录。于是 $\Delta U = M - J - R$ ， $\Delta D = M - J$ ，而回款给出 $\Delta A = R$ ，所以 $\Delta(U + A - D) = 0$ 。金额为 a 的赔付给出 $\Delta A = \Delta D = -a$ 与 $\Delta U = 0$ 。增额在被计入的账户之间转移。其他转移保持这一投影。 ■

FUEL 使用单独的账务。每份托管的最大值等于 Held 加上累计的 Rewards、Burned 和 Refunded。全局守恒的总和计入未花费的 FUEL、普通账户、当前的 Held、实际的奖励余额和累计销毁。唯一的阶段与义务标识符只转移价值一次；已退还的输出和已入账的奖励不会再通过累计的托管字段重复计算。这同时适用于用户付费的费用和经授权的组织代付费用。

守恒固定的是总量，而不是谁持有。下一个结果表明，延迟不改变最终持有人。

定理 3 (延迟中性): 令 $\mathcal{T}$ 为一个有限的付款集合，满足：(i) 没有两笔付款消费同一实例或同一费用输入；(ii) 关系“消费由其创建的输出”是无环的；(iii) $\mathcal{T}$ 中付款的每个证书输入所认证的，都是 $\mathcal{T}$ 中某笔付款的输出。令 $\mathcal{U}$ 为一个有限的经授权备付增额集合。从一个干净的创世状态出发，没有覆盖、promise 或义务记录，且 CAL 的 Reserved 与 Spent 为零。令 ρ_0 按与 (ii) 一致的顺序执行 $\mathcal{T}$ ，并应用 $\mathcal{U}$ ；令 ρ 为任意由 $\mathcal{T}$ 中的付款、 $\mathcal{U}$ 中的增额以及对已到期 Open 义务的赔付按任意顺序组成的执行。假设在两次执行中，每笔付款与每次增额都恰好成功一次。则在 ρ 与 ρ_0 结束时：

- 1) 最终的、未花费的 CAL 实例相同，所有者与金额也相同；
- 2) 每个备付账户持有相同的 CAL 余额；
- 3) 每个 CAL 授权额度有相同的授权，且 $\text{Reserved} = \text{Spent} = 0$ ，每项义务为 Fulfilled 或 Recovered，且每张证书满足 $\text{Paid}_c = \text{Rec}_c$ 。

费用与 FUEL 不作比较。

证明: 持有人。由 (i)，两次执行消费相同的实例。在 ρ 中，在其来源执行之前被消费的输出会获得一项义务；来源执行时，该输出要么是 Fulfilled，其创建记录已被消费，要么是 Recovered，没有创建记录。无论哪种情形，它都已被消费。没有被 $\mathcal{T}$ 中任何付款消费的输出，在两次执行中都被创建为最终的、未花费的。因此，两次执行中未花费的实例都是创世输出与未被 $\mathcal{T}$ 消费的 $\mathcal{T}$ 的输出。

备付金。对备付账户 x ，令 $\hat{A}_x$ 为其余额加上由 x 发行的 Repaired 义务的金额。赔付使 A_x 减少、Repaired 总额增加相同金额；由引理 3，回款在同一账户上作相反的变化；创建或履行义务不改变这两项。如同定理 1，只有增额、赔付和回款会改变备付余额。因此在每个前缀上 $\hat{A}_x = A_x(0) + \tau_x$ ，其中 τ_x 为迄今记入 x 的增额总额。义务只存在于在其来源执行之前被消费的输出。由 (iii)，该来源在 ρ 中执行，其执行使该义务变为 Fulfilled 或 Recovered。所以结束时没有任何义务为 Open 或 Repaired， $A_x = A_x(0) + \tau_x$ ，而这也是 ρ_0 中的值，其中没有发生赔付。

使用量。每张已登记的证书都因某个缺失输入而被使用，故由 (iii)，其来源在 ρ 中执行。该执行关闭它的全部 promise，故 $\text{Remaining} = 0$ ，且由引理 3 得 $\text{Paid}_c = \text{Rec}_c$ 。因此 $\text{Reserved} = \sum \text{Remaining} = 0$ ， $\text{Spent} = \sum (\text{Paid}_c - \text{Rec}_c) = 0$ 。授权只通过 $\mathcal{U}$ 变化。 ■

中间状态不同：在已赔付的来源待执行期间，其发行者的余额低于赔付金额，这由定理 1 覆盖。该定理并不主张激励相容，也不涉及永不执行的来源，其已赔付的金额始终是其发行者的净损失。

D. 授权修复与表示独立性

引理 5 (受限的修复授权): 每个新生效的修复条目都关闭其唯一的、已到期的、在执行前状态下为 **Open** 的义务。即使适配输出是对抗性的，一次成功的修复也不会改变其授权资金位置之外的任何交易字节。

证明: 令 N 为 RSA 模数, e 为其公开指数, r 为 $\mathbb{Z}_N^*$ 中的一个 **opening**。带域分隔的代表元 $\text{FDH}(ctx, m)$ 绑定消息及其修复上下文。关系

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

确立的是相容性，而不是新的批准；已公开的 **opening** 比值可以在同一消息关系下被复用。公共守卫独立地导出位置、发行者、金额和备付金扣款标识，检查当前的基础修订与正文摘要，并验证所有非目标字节与完整区块承诺。无效候选不留下任何变更。精确的成功重放是无操作；重新打包的命令会遇到共享的义务终态。经济效果与经授权的修订 **Task** 一起提交。适配器既不写账本状态，也不能伪造委员会认证。该论证使用其经验证的输出，而不是新的门限参与的证据。 ■

令 π_{econ} 保留公共资产、消费、覆盖、义务、授权额度使用量和费用状态，省略修订与安装元数据。

引理 6 (批等价与安装独立性): 在其提交边界处，一个被接受的批具有如下经济效果：它等同于从同一状态出发、在同一时间、且无中间命令地依序执行其各单项赔付步骤。对同一公共历史，合法的物理安装调度具有相同的经济投影。

证明: 唯一的义务与位置按规范排序。每一步读取共享的 **overlay**，包括先前的账户、**Pending** 和费用变更。失败会丢弃所有步骤；成功则把它们的汇总效果与一个修订一起提交。**Base** 是修订号；**Previous** 与 **Next** 认证正文摘要。新修订为 **Base+1**。

Canonical 读取已提交的逻辑修订 ρ_ℓ 。一个精确的 **Task** 授权修订 $r < \rho_\ell$ 处的正文与分片。若 $\rho_p < r$ 则推进安装，若 $\rho_p = r$ 则检查完全相等，把 $\rho_p > r$ 视为已涵盖而不回滚。它不改变任何经济变量。队列进度只跨越已完成的条目：在 A_1 处安装 A_2 不能跳过中间的 B_1 。因此，对同一公共前缀上的合法调度 μ_1, μ_2 ,

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

批命令与单项命令的历史可以有不同的应用哈希；对同一原始命令历史的确定性重放在每个高度重现其应用哈希。修复保持固定的交易字段、授权、输出身份、后继引用以及完整区块身份。

对于从同一本地状态、配置和游标出发的跟随者，只要两种表示都能解码，相同的固定付款字段与经认证的结果就会导致相同的经济更新。可变的 **Funding/opening** 不决定 **Paid**、**Pending** 或费用效果。修复命令被完整认证；执行结果承诺认证其逐条目效果与费用输出。这把经济观察与对最新可变历史正文的独立验证区分开来。 ■

E. 受保护的续花与进展

定义 **FreshRecv**(w, o, c) 为一名诚实所有者对认证收款的接收，且该所有者此前没有授权消费该实例。新鲜性是一个历史谓词，不能仅由成功收款推出。

定理 4 (受保护的续花): 在 **FreshRecv** 之后，已认证的输出不能在所有者授权的情况下被重新解释或被消费。一笔已认证并已公共执行的后继付款，要么消费一个最终来源，要么原子地建立一项有资金支持的直接义务。来源关闭与修复保持其已确立的本金效果，无需后继付款重新签名或重新执行。

证明: 引理 2 固定所有权并排除冲突消费。原子执行把每次缺失消费与其义务及输出耦合。定理 1 提供资金支持；定理 2 核算关闭与迟到来源。引理 1 确定已执行的历史，而引理 5-6 限制之后的变更。对本地/公共联合转移的归纳把这些性质组合起来；**INSTALL** 保留本地消费效果，但不改变公共资产。修复可以改变 **Pending**、费用和备付余额，而不重复后继付款的本金。 ■

续花需要响应的诚实法定人数、可得的证据、足够的费用与 **Worker**/工作量容量，以及公平的投递与纳入。修复还需要公共时间推进以及可得的适配贡献。部分批准与未使用的 **promise** 可能占用容量；超时本身绝不授权解锁。这些是进展条件，而不是普遍的完成期限。

数据可得性与担保人风险。有三个问题需要分开。安全性：定理 1 为每一张曾签发的证书提供资金支持，无论其来源是否被提交。进展：当某个诚实中继持有其完整的提交材料（交易、准入向量、组织授权、直接输入证书），且其余的费用、容量、法定人数和公平调度条件成立时，来源可以取得进展；投递时间没有界。因此，截止时间可能因拥塞或短暂中断等常规原因而过去。此

后，只要适配贡献可得，赔付即可提交：发行者垫付缺失的本金，并在来源执行时得到回款。当该付款集合的所有来源都执行时，定理 3 表明最终持有人与备付金余额不变。经济性：成员批准端点认证所有者、预算和输入，但不认证调用者的角色，因此任何客户端都可以充当收集器。如果该客户端从不安装或提交已汇集的材料，且没有其他中继获得它，则来源保持未执行；一旦已被赔付，其发行者就把该金额作为未回收的损失承担。备付金规模应反映这类来源的预期比例；费用并不被定价为针对它们的保险。

VI. 实现与评估

Next 的评估把收款方侧的续花与有资金支持的关闭及保持身份的修复联系起来。我们先考察相互依赖的付款，然后依次考察来源故障、修复成本、组织边界和运行容量。这些测量检验第 IV 节的机制在第 III 节模型下的表现；第 V 节则在所述模型下确立它们的安全性质。

实现与测量。 Go 实现使用原子的状态 overlay、独立的区块跟随者，以及经修改以强制执行完整区块身份和经授权的历史安装的 CometBFT v0.38.26。门限 RSA 适配使用 CIRCL v1.6.5，并采用可信离线 dealer 初始化。网络实验运行在一台 Mac Studio 上，配置为 M4 Max、16 个 CPU 核、64 GiB 内存、macOS 26.6.2 和 Go 1.27.1。四个委员会进程，加上每个组织的四名成员和一个网关，共得到九个或十四个服务进程。延迟/容量试验主要使用内存的成员与委员会后端，以及采用 NoSync 的 bbolt 钱包；签名与原子状态更新保持启用。来源修复与批集成试验使用磁盘支持的委员会应用状态和 BlockStore。每次运行都从全新创世开始。当前的续花和回款试验使用最终的用户 FUEL。存储、配对背景和短时吞吐对照使用经授权的组织 FUEL 代付。归档的续花、运行区间和批集成系列使用经授权的费用代付。补充材料记录每个系列的构建与设置；批集成单独于较早的结构测量进行报告。

已认证收款的延迟通常从钱包实际发出第一个 HTTP 请求开始，到收款方验证并原子地本地记录之后结束。资金/来源故障驱动包含少量发送前的构造，并在各自的系列内比较。公共完成与成员完成是观察端点，而不是内部提交时间戳。完成吞吐包含最终的后台排空。跨站点 ACK 计时使用发送方的单调时钟，并包含返回路径。

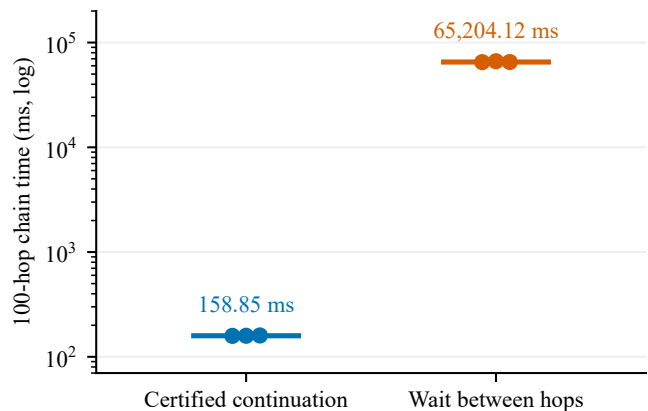


图 4. 100 跳链到最后一个已认证收款的耗时。圆点为每种模式三次独立运行；水平线标出其中位数。对数时间轴在两种模式中包含相同的端点。只有等待基线要求相继付款之间进行公共确认。

A. 无需逐跳结算的持续花费

一条 100 跳的付款链到达最后一个已认证收款的中位时间为 158.846 ms，而当相继的各跳等待公共确认时为 65.204 s。三个钱包轮流转移 100 CAL，每笔付款都由其前驱的输出在线构造，并用各自的最终 FUEL 支付费用。每种模式重复三次，在九服务内存配置上共产生六次运行和 600 笔付款。两种模式都在最后一个已认证收款处结束；等待基线则在各跳之间插入公共确认等待。委员会的提交、flush 和 gossip 设置分别为 250、10 和 10 ms。在这些设置下，等待基线中的公共执行及其观察每跳耗时约 0.65 s；认证续花移除了这一跳间等待。

图 4 给出六次整链耗时。快速运行耗时 158.492–159.905 ms。全部 297 个快速模式的后继请求都消费前一个已认证的输出，并在其已认证收款之后、但在观察到其公共完成之前发出。一个单输入、单输出的 TXCer 为 779 字节；请求在第一跳为 1,979 字节，之后为 2,766 字节。对这一固定的交易形态，直接证据的大小在全部 100 跳中保持不变。这些观察表明，在线的相依花费不会在收款方的关键路径上插入公共确认等待。

同步成员提交。 在九服务磁盘配置上运行三对 100 跳链，钱包保持 NoSync，用户自付 FUEL，共识参数仍为 250/10/10 ms。每对仅成员存储不同 (NoSync 或同步提交；顺序 N/S、S/N、N/S)，每对之后再运行一条同步提交的等待链。中位耗时分别为 681.127 ms (NoSync 续花)、4.471 s (同步续花) 和 65.013 s (同步等待)。因此，保留成员同步提交时，认证后续花仍比逐跳等待快 14.5 倍。全部 900 笔付款完成，各委员状态一致；补充材料列出逐轮数据。

在归档的交付门槛构建上，在匹配的提供负载下，移除认证后的交付门槛同样缩短了续花。带门槛的变体等待三名成员安装完整材料，或等待经认证的公共成功，以先发生者为准；它不收集第二个签名法定人数。在每秒 200 笔、持续 30 s 的负载下，每种模式的三次配对运行各自发送并完成 6,000 笔付款。运行级收款 P50 的中位数，在立即交付时为 1.300 ms，带门槛时为 2.167 ms，降低 40.0%。另外的 100 跳试验把链的中位耗时从 287.983 降到 200.356 ms，即降低 30.43% (图 5)。这一收益去掉了一次复制/完成等待，同时保持相同的三签名授权。

B. 有资金支持的关闭与来源回款

来源回款矩阵。九次运行使用种子 23、37 和 59，被扣留父交易的比例为 0%、1% 和 5%。每次运行以每秒 20 个两跳单元的速率提供负载，持续 60 s (1,200 个单元，2,400 笔付款)，对应固定的 60,000-CAL 备付金，用户支付 FUEL，且没有增额。九服务部署使用磁盘支持的委员会应用状态和 BlockStore、内存的成员与网关，以及 NoSync 钱包。被扣留的父交易在其赔付已提交并已在全部四个副本上安装之后才被释放，因此每个被扣留的来源在 30 s 的公共时间截止之后被赔付，并在其后执行。

全部 21,600 笔付款都已关闭。216 次赔付（每次 1% 运行 12 次，每次 5% 运行 60 次）之后都有回款。每次运行结束时，没有任何义务为 Open 或 Repaired，缺口为零，净 Spent 与 Reserved 为零，备付金恢复到 60,000 CAL (表 II)。图 6 给出三次 5% 运行的轨迹。采样到的最低备付金在 1% 时为 59,800–59,900 CAL，在 5% 时为 59,600 CAL；它反映的是驱动器的释放调度，而不是备付金规模的结果。正常子付款的运行级收款 P50 为 1.185–1.251 ms，被扣留的父交易及其子付款分别为 1.171–1.207 和 1.176–1.190 ms。截止时间之后，委员会提交被观察到的时间中位数为 1.003–1.254 s，在全部四个副本上完成安装的时间为 1.257–1.761 s (P95 最高 2.015 s)，其中包含 250-ms 的观察轮询。每笔正常付款花费 94 FUEL，每次修复增加 5；奖励、销毁和退款之和等于费用上限，Held 为零。

账本级回归直接覆盖了各种到达顺序：对一条四跳链的全部 24 种到达顺序，分别在有和没有到期赔付的情形下，在三种发行者序列（单一组织、交替的两个组织、以及首个发行者重复出现的三个组织）下，共得到

表 II

来源回款矩阵的收尾审计。每列在三个种子上相同；每次运行有 2,400 笔付款。备付金数值为 CAL，费用数值以千 FUEL 计，费用上限为各笔付款最大值之和。

被扣留的父交易	0%	1%	5%
修复 / 回款	0	12	60
赔付 / 回收的 CAL	0	1,200	6,000
最终备付金	60,000	60,000	60,000
未关闭 / 净 Spent	0	0	0
费用上限合计	2,400	2,400	2,400
奖励	201.600	201.660	201.900
销毁	24.000	24.000	24.000
退款	2,174.400	2,174.340	2,174.100
剩余 Held	0	0	0

144 条轨迹。另有 48 条拆分-合并轨迹，在两种赔付模式下改变一个 40/60 拆分、两个分支及其合并的全部到达顺序。这 192 条轨迹中的每一步都保持独立计算的资产投影守恒，且每条轨迹结束时各发行者的备付金均已恢复，Spent 与 Reserved 为零，Paid = Rec，已消费的输入仍保持已消费。

另外两个十四服务的控制实验区分了永久损失与延迟回款。当父交易始终不到达时，孙付款保留其 100 CAL，而负责的发行者的备付金保持低 100 CAL。当跨组织链以孙付款先到达时，中间来源关闭其自身的直接义务；根发行者被赔付的 100 CAL 在根之后执行时得到回款。两种情形中四个委员会的状态都一致。缺席父交易自身的费用与工作量预留可能保持未决。

配对终态检验与干扰。三对十四服务运行按正常/修复、修复/正常、正常/修复的顺序，使用最初的来源回款构建，即批结果标签分离之前的构建，委员会与成员数据库在磁盘上，成员与钱包为 NoSync，每次以每秒 100 笔、持续 40 s 发送 4,000 笔付款。与此同时，另有四个两跳演示，其来源要么被正常送达（正常运行），要么被扣留、在 30 s 后被赔付并随后回款（修复运行）。六次运行中的 4,000 笔付款全部完成。各所有者角色最终持有的 CAL (30,000 与 830,000) 以及两个备付金余额 (10^{12}) 在六次运行中完全一致，且修复运行记录了 400 CAL 的赔付和 400 CAL 的回收。正常与修复运行的收款 P50 分别为 1.070/1.127、1.089/1.108 和 1.083/1.097 ms，P95 分别为 26.84/59.23、29.49/58.44 和 27.41/55.15 ms；各次运行的完成速率为 97.2–97.8 笔/秒。最终分配与定理 3 一致。收款 P50 比配对的正常值高 1.3–5.3%，P95

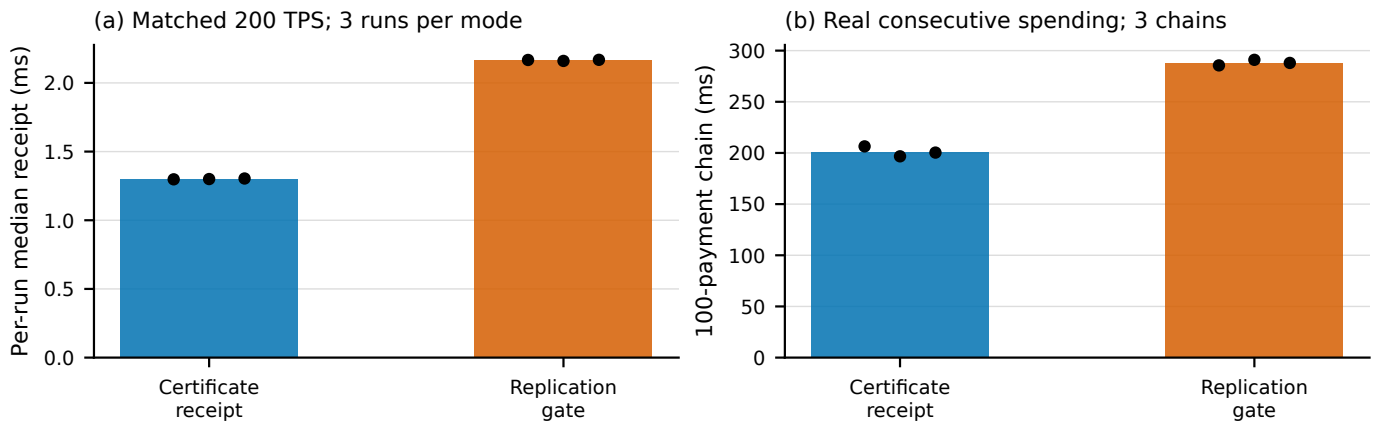


图 5. 交付门槛的影响 (ms)。左：每秒 200 笔负载下的运行级收款 P50，每次运行 6,000 笔付款，三次配对重复。右：每种模式三条 100 跳链的整链耗时。柱为三次运行值的中位数；圆点为各次运行。

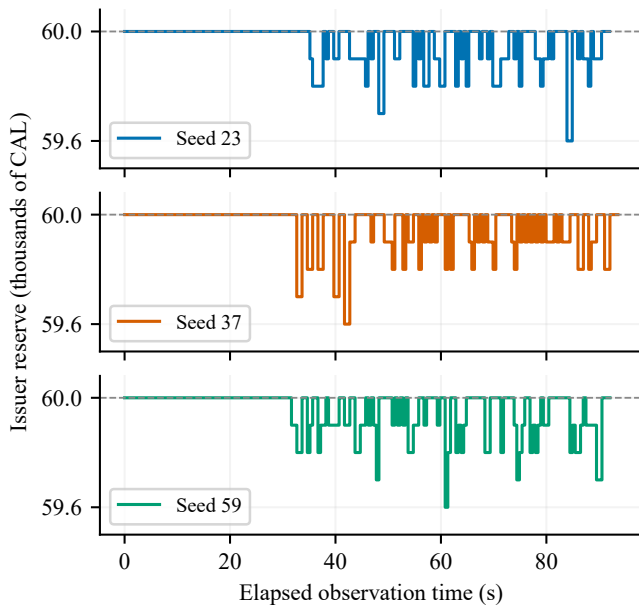


图 6. 三次 5% 运行（各有 60 个被赔付的来源）的备付金采样轨迹。回款使备付金恢复到初始的 60,000 CAL。纵轴限制在 59,600–60,000 CAL，以显示临时占用。采样反映驱动器的释放调度。

为其 1.98–2.21 倍。因此在此配置下，并发的修复会影响收款的尾部；这些配对运行并不能分离出造成这一点的阶段。

存储对照。在九服务配置（委员会在磁盘上，钱包为 NoSync）上，六次各 1,000 笔、每秒 100 笔的运行，仅成员存储不同。成员为 NoSync 时，收款 P50 为 1.002–1.035 ms，P95 为 33.97–40.62 ms，含排空完成 93.11–93.93 笔/秒。成员采用同步提交时，P50 为 43.19–51.85 ms，P95 为 80.98–96.56 ms，完成 90.99–92.72 笔/秒。全部付款均完成。这隔离出同步成员提

表 III

实际的资本增加。每个条件有一次 300-s 运行，从 3,600 CAL 开始，完成 12,000 笔付款。两列均为 CAL；每次增加都从一个外部出资账户扣款。

负载	最终备付金	增加的资本
恒定：20 单元/秒	14,400	10,800
阶跃：10 到 30 单元/秒	21,600	18,000
父交易延迟 3 s	23,300	19,700

交的代价；钱包仍为 NoSync，因此这是在第 III 节执行模型下的成员提交对照。

在较早构建的资本系列中（其中没有发生赔付），真实的外部资本支撑了更长时间的运行。单独的 300-s 实验为每名成员使用一个 Worker，并提供充足的用户 FUEL，每个条件运行一次。起始为 3,600 CAL 的恒定速率、速率阶跃和 3-s 延迟父交易负载，各自通过表 III 中的转账完成 6,000 个两跳单元。控制器使用实验所声明的提供流量。相应的固定高备付金对照均能完成；固定 3,600-CAL 的对照关闭了 6,000 个单元中的 46 个，并保留部分批准。补充材料给出控制器以及全部七个条件，包括其准入限制。

C. 在保持后继付款的同时共享修复工作

同块批处理共享表示工作，而不合并被赔付义务的身份。在 Windows amd64 与 Go 1.27.1 上的结构性微基准，把 $k \in \{1, 2, 4, 8\}$ 个缺失输入放入同一个历史区块的同一个分片。在五次重复中，单项修复需要 k 条命令和 k 次分片适配，而一个批只用各一次；输入适配仍为 k 次。在 $k = 8$ 时，编码后的命令总字节从 127,336 降到 2,752（图 7）。

分散对照把批处理与紧凑编码区分开来。八个独立的目标区块仍需要八条命令和八次分片适配，而字节数从 30,008 降到 5,440。在 $k = 1$ 时，两种格式都执行一次分片适配，编码分别为 3,751 与 680 字节。因此，紧凑字段解释了字节减少的一部分，而共享的区块表示解释了分片适配次数的减少。这些是编码后的命令大小，不是总网络流量或总密码学工作量。

跨层回归执行检查后继付款的保持与经济一致性。对于一笔使用 40/60 输入的付款及其已执行的后继付款，来源修复保持原始授权、输出身份和完整区块身份。在固定的前状态、时间和条目顺序下的比较，给出与逐项赔付相同的经济投影。重放同一原始公共历史会重现每个高度的应用哈希。这些回归还涉及跨分片故障、原子的批拒绝、重复命令和交错目标的安装。

一次归档的十四服务、磁盘支持的批集成运行，把四个实际缺失的来源合并为一个 1,568 字节的批，其 BatchID 在全部委员会副本上一致。处理完迟到父交易之后，以每秒 100 笔提供的后续 1,000 笔付款全部完成。收款 P50/P95 为 1.048/26.531 ms；总完成时间为 11.120 s，含观察与关闭。缺口与发件箱排空，委员会状态哈希一致。结构性测量隔离出表示成本；这次集成运行则检验修复以及部署服务之间后续付款的处理。

D. 跨注册组织的续花

跨组织续花沿用同一条证书路径，其延迟反映所注入的通信距离。在补充材料标明的归档构建上，双组织套件在 24 次运行中完成 342,000 笔付款。两个钱包进程各持有 32 个身份，分布在 64 条通道上；每个组织提供每秒 50 笔付款。延迟比较使用十跳的同组织/跨组织负载，注入的 HTTP RTT 为 0、20 和 100 ms，每个条件重复三次。每个代理方向增加一半的 RTT；组织内部的认证和委员会 P2P 不注入延迟。

对于共同的前 120-s 队列，三次运行级跨组 ACK P50 的中位数分别为 1.632、21.658 和 101.967 ms；同组的值在 1.615–1.744 ms 之间（图 8）。ACK 包含返回路径，但收款方可以在接受之后即发起其后继付款，而无需等待该 ACK 到达前一个发送方。在 100-ms 跨组 RTT 下，三次 300-s 运行各完成 30,000 笔付款；待决峰值为 106–110，并且在排空时没有持续的尾部增长。该实验在一台物理主机内隔离出 HTTP 路径延迟。

E. 运行容量与服务可用性

当前构建的独立输入吞吐。三次各 30,000 笔付款的运行，在九服务内存配置、NoSync 钱包以及经授权的组织 FUEL 代付下，以每秒 2,000 笔为目标。成员进程使用 200% 的 Go GC 目标和 16 个运行时处理器。全部付款完成，四个委员会状态一致。含排空的完成速率为 1,750.27、1,820.91 和 1,887.31 笔/秒，历时 15.9–17.1 s，收款 P50 为 2.646、2.842 和 2.824 ms，P95 为 47.9、49.1 和 48.0 ms。驱动器记录到 6,551–7,570 笔付款必须等待一个快速收款准入名额，且 P95 的派发滞后为 179–229 ms，因此提供负载并非均匀地维持在每秒 2,000 笔。这些是短时的完成测量；补充材料另行标明 150,000 笔付款系列的构建与时长。

在归档构建上，冗余的认证与已安装的证据，使服务在所规定的故障下得以维持。在每秒 200 笔的九次运行中，一名成员被延迟或暂停 30 s，完成 216,000 笔付款。带有一份完整已安装副本的网关丢失对照显示，备用提交发生在 INSTALL 之后 2.503–2.508 s；进展需要一份幸存的完整副本。冲突对照既不产生相互冲突的双重 QC，也不产生重复的本金/费用效果，而部分批准仍是容量成本。

补充材料保留冻结构建的映射和完整的逐次运行记录，包括多身份的 CPU/HTTP 成本，以及与积压分开的历史内存增长。其 2,048 个身份共用客户端，而不是代表 2,048 个进程。它还记录了一个单独配置的 LND 部署参考，其完成端点和持久化设置各不相同。

VII. 结论

Next 通过把已认证收款与公共执行及来源解决分离，支持持续的 UTXO 支付。其分层架构使收款方在证据复制于后台进行的同时即可请求后继付款。直接发行者义务使缺失来源的执行有真实的备付金支持；累计账务覆盖隐藏证书，来源迟到的回款向发行备付金偿还。对于来源全部执行的集合，延迟保持本金的最终分配不变。经授权的修复随后保持固定的付款字段与完整区块身份，同时批处理表示工作，并允许安装推进到已提交的修订。原始历史的重放与安装时机无关。安全性分析把这些机制与唯一消费、有资金支持的负债和资产守恒联系起来。对依赖付款的测量以及磁盘支持的修复执行展示了已实现的路径。在注册组织模型内，Next 使收款方续花成为前台操作，而来源解决成为有资金支持、可审计的后台责任。

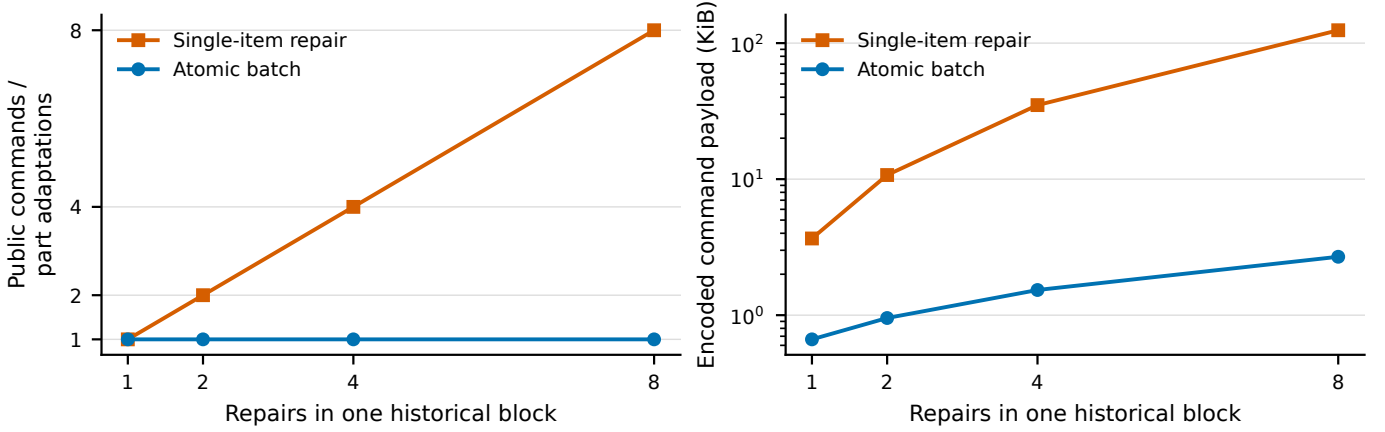


图 7. 同块、同分片的修复成本随义务数 k 的变化。两个面板分别显示命令/分片适配次数，以及编码后命令载荷总量 (KiB, 对数刻度)。每种配置五次重复给出相同的结构性测量。输入适配仍为 k 。

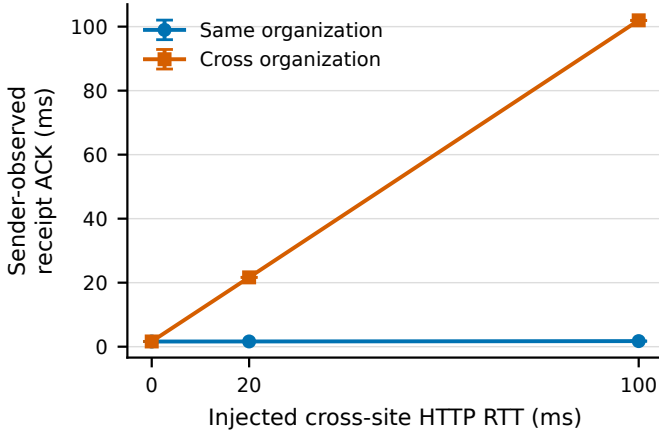


图 8. ACK 延迟 (ms) 随注入的跨站点 HTTP RTT (ms) 的变化。每个点为前 120-s 队列中三次运行级 P50 的中位数；误差条跨越这些 P50 的最小值与最大值。总提供速率为每秒 100 笔。委员会 P2P 未加延迟。

致谢

ChatGPT 与 Claude 辅助起草和修订摘要、正文及中文阅读版；Codex 辅助基于源码的审阅、修订、LaTeX 组装与绘图脚本。实验图表由所记录的测量数据生成。

参考文献

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>
- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [7] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [8] V. Mavroudis, K. Wüst, A. Dhar, K. Kostianen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [9] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [10] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.
- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [12] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [13] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [14] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.

- [15] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, "Reparo: Publicly Verifiable Layer to Repair Blockchains," in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>
- [16] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, "A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content," in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [17] X. Chen and Y. Gao, "CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types," 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [18] E. Buchman, J. Kwon, and Z. Milosevic, "The latest gossip on BFT consensus," 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>